

SOGR: SEMANTIC OBJECT GRAPH RUNTIME

A Structured Incremental Architecture for Long-Context Language Modeling

Zeping Tu

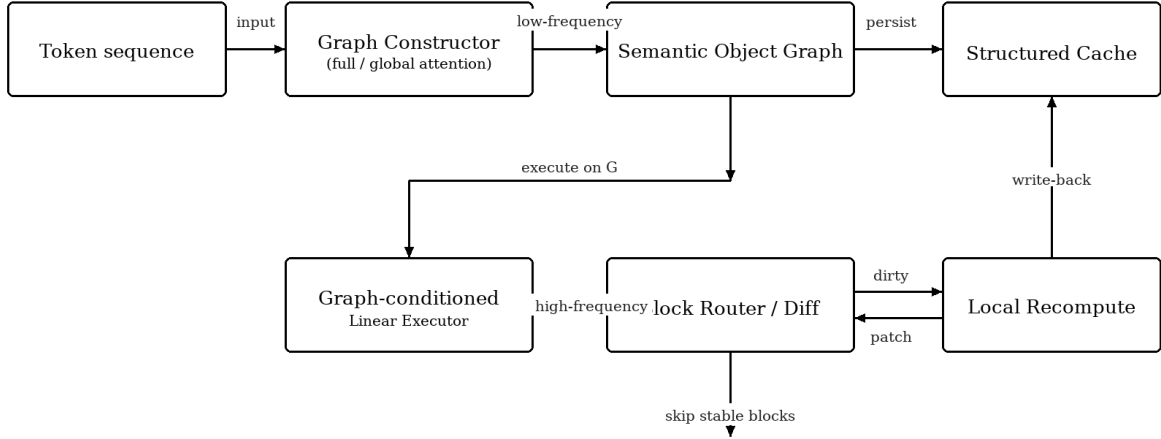
Frontend Engineer

Independent Research Draft · Version 0.3 · September 2026

Abstract. This paper proposes SOGR, a language-model architecture that treats semantic dependency structure as an explicit computational intermediate representation rather than forcing every layer to rediscover relationships over a token sequence. The central hypothesis is that a relatively infrequent global-attention stage can construct or repair a Semantic Object Graph (SOG), after which graph-conditioned linear state propagation performs the high-frequency computation. A structured cache stores node, edge, and block states; block-level change detection provides a coarse routing mechanism analogous in spirit to dirty-region propagation in reactive user-interface systems. When a local variable, entity, or relation changes, only its dependency subgraph is recomputed, while unrelated state is retained. Because the cache is an object graph rather than a dense per-layer key-value tensor, a further deployment hypothesis follows: the portable part of the cache can live on the client, while the server retains model weights and performs graph-conditioned computation. Model-specific hidden states are treated as ephemeral materializations; structure and a compact canonical embedding can be lifted into a new model’s coordinate system instead of attempting to translate raw KV tensors. The proposed architecture combines five ideas: (1) low-frequency global graph construction, (2) high-frequency graph-conditioned linear execution, (3) periodic global graph repair, (4) hierarchical block-level skipping, and (5) a three-layer cache that separates portable structure from model-bound activation. We derive a conditional complexity model showing that the method can outperform dense attention only when graph construction is amortized, dependency density remains sufficiently sparse, and graph-maintenance overhead is bounded. The portable-cache claim is likewise conditional: lifting a stored graph must remain cheaper than full prefill, and the graph must remain small enough to move off the accelerator. This manuscript is a research proposal together with a CPU-laptop baseline. It is not a report of GPU-scale training. No claim is made that SOGR has been shown to match full-attention quality at production scale. The intended contribution is an explicit architectural hypothesis, a falsifiable experimental program, and the current floor of what can be measured on a non-gaming notebook.

Keywords: semantic object graph; incremental computation; structured cache; portable working memory; linear attention; sparse computation; block routing; long-context language modeling; model-agnostic intermediate representation

Semantic Object Graph Runtime (SOGR)



Global construction is low-frequency; graph propagation and local recomputation are high-frequency.

Figure 1: Overview of the proposed SOGR runtime. Global attention is used as a low-frequency graph-construction operator; graph-conditioned linear operators execute repeated updates; block routing and structured caching skip unnecessary recomputation.

1. Introduction

Transformer-based language models represent context primarily as a sequence of token states and repeatedly apply attention to infer interactions among those states. This design is powerful because it makes few assumptions about which relationships matter. Its cost, however, is that relationship discovery is repeatedly entangled with information propagation. Even when the effective dependency pattern is sparse, a conventional dense attention operator does not expose that sparsity as a first-class computational object.

A different design question is therefore proposed here: what if the model first constructed an explicit, reusable representation of language relationships, and subsequent computation operated on that representation rather than rediscovering the relationships at every layer? The research direction originates in frontend engineering. Reactive frameworks such as React and Vue separate a structured view (the virtual DOM or a reactive dependency graph) from rendering: a local state change dirties only the dependent subtree, while unrelated regions are skipped. The analogy is not that natural language is a DOM. The transferable idea is the runtime contract: compile structure once, execute incrementally, persist the object rather than the pixels. Compiler intermediate representations and abstract syntax trees supply the same split between structure and execution. SOGR asks whether a language model can be organized around a Semantic Object Graph in that spirit.

SOGR formalizes this idea as a four-stage runtime. First, a global attention module constructs a Semantic Object Graph (SOG). Second, graph-conditioned linear operators propagate state along the graph. Third, after a configurable number of linear steps, a global attention module repairs or extends the graph. Fourth, hierarchical block summaries decide whether a region is stable enough to skip fine-grained computation. A structured cache stores the resulting state and invalidation metadata.

If that cache is small and structurally explicit, the runtime boundary can move. Conventional KV cache is bound to a serving process because it is large and expressed in a model’s private layer and head basis. A semantic object graph can instead be stored locally as a working-memory object. The server then primarily sells computation: it lifts the portable graph into the current model’s hidden space, executes only the invalidated subgraph, and returns a versioned patch. Changing models becomes a conversion of this structured object, not a translation of raw KV tensors.

The proposal is deliberately narrower than a claim that sparse attention or linear attention is sufficient. Existing efficient architectures already demonstrate that linear-time sequence processing and hybrid attention can be practical. The proposed novelty is the organization of computation: relationship discovery becomes an explicit, cacheable state; computation becomes an incremental graph update problem; and the portable residue of that state can, in principle, leave the accelerator.

2. Design Motivation

The first-generation efficiency idea considered in this research direction was token selection: identify salient positions and discard the rest. Such methods can reduce computation, but they still treat the token sequence as the fundamental object. The proposed architecture instead asks whether the underlying dependency structure can be made explicit before deciding what to compute, in the same way a frontend framework diffs a component tree rather than repainting the entire document.

Consider the sentence “Alice bought a laptop in Hangzhou because her old computer failed.” A token-only representation requires the model to infer that “her” refers to Alice, that “old computer” is distinct from “laptop,” and that the causal clause explains the purchase context. A structured representation can expose these as candidate nodes and edges. Importantly, the graph is not required to be linguistically perfect; it may contain weighted, uncertain, or revisable edges.

The key engineering principle is therefore: do expensive global relationship discovery less frequently, and reuse the discovered structure for many cheaper propagation steps. This is analogous to compiling a program into an intermediate representation and then repeatedly executing the compiled structure, rather than reparsing the source code for every operation.

A second principle follows from the same compilation analogy. Source and IR can live with the user; the compiler or runtime can be remote and stateless with respect to long-lived context. Token-level KV cache cannot play that role: it is both too large to shuttle cheaply and too tightly coupled to one set of weights. A sparse object graph is a candidate for that role only if structure is stored separately from model-specific activations.

3. Semantic Object Graph

Let a sequence of L token embeddings be $X = \{x_1, \dots, x_L\}$. SOGR introduces a graph $G = (V, E, A)$, where V contains token, phrase, entity, event, or latent semantic objects; E contains typed dependency relations; and A stores relation confidence or routing weights. The graph may be hierarchical: token-level nodes can belong to phrase nodes, which belong to sentence or block nodes.

A minimal graph node can be represented as $N_i = (h_i, c_i, \text{type}_i, \text{span}_i, \text{parent}_i, \text{version}_i)$, where h_i is the current model hidden state, c_i is an optional canonical semantic vector, and version_i supports cache invalidation. An edge is $E_{ij} = (r_{ij}, w_{ij}, \text{version}_{ij})$, where r_{ij} is a relation type and w_{ij} is a learned or inferred strength. Unlike a fixed syntactic parse, the graph is allowed to be revised by later global passes. The tuple $(\text{type}, \text{span}, \text{parent}, r)$ is intended to be the model-agnostic skeleton; h_i is not.

The important distinction from conventional sparse attention is that the graph is intended to become a persistent computational state. A sparse attention mask is usually a per-layer routing decision. In SOGR, the graph can persist across multiple linear-execution stages and can therefore serve as an index for incremental recomputation. Persistence across serving sessions and across models is a stronger claim and is isolated in Section 5.

4. Architecture

The architecture consists of four principal operators: Global Graph Constructor (GGC), Graph-conditioned Linear Executor (GLE), Periodic Graph Repair (PGR), and Hierarchical Block Router (HBR).

4.1 Global Graph Constructor. A full-attention or otherwise high-capacity module examines the current state and proposes graph nodes and weighted relations. The expensive operation is intentionally low-frequency. It is not assumed that one graph remains correct indefinitely.

4.2 Graph-conditioned Linear Executor. Given G , a linear operator propagates state only along graph-supported relations. One generic formulation is

$$h'_i = U h_i + \sum_{j \in N(i)} w_{ij} V_{r_{ij}} h_j,$$

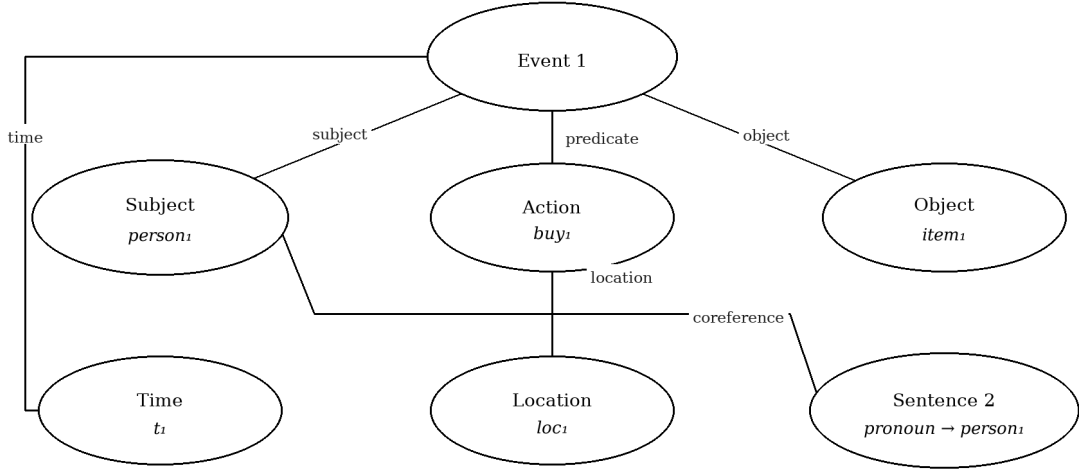
followed by a gated state update. The implementation can use gated linear attention, delta-rule memory, SSM-like state transitions, or another linear-time mechanism. The architecture therefore does not depend on one particular linear-attention formulation.

4.3 Periodic Graph Repair. After K linear execution steps, a global module re-evaluates the graph. It may add edges, remove stale edges, alter weights, split nodes, merge nodes, or simply confirm that the current structure remains valid. This is the mechanism intended to address the finite-state and retrieval limitations of purely linear models.

4.4 Hierarchical Block Router. Tokens are grouped into blocks. Each block maintains a compact summary z_b and a change indicator Δ_b . If a new query or upstream state has sufficiently low interaction with a stable block, the block can bypass fine-grained computation. The routing unit is intentionally block-sized because contiguous block execution is more hardware-friendly than arbitrary token-level sparsity.

4.5 Structured Cache. The cache stores node states, edge states, block summaries, and dependency versions. A change to node i increments its version and invalidates only descendants or dependents reachable through the graph. Cache validity is therefore structural rather than purely positional. Internally, this cache is the incremental-computation store. Externally, only a subset of it is a candidate for client-side persistence, as defined next.

Example Semantic Dependency Graph



A change to $person_1$ invalidates only dependent nodes/edges rather than the entire sequence.

Figure 2: Example semantic dependency graph. The graph stores typed relations and can propagate invalidation from a changed semantic object. A change to $person_1$ invalidates only dependent nodes/edges rather than the entire sequence.

5. Portable Structured Cache

This section is a deployment hypothesis, not a required property of the four-stage runtime. It asks whether the same IR that enables incremental recomputation can also become a portable working-memory object.

5.1 Why token KV cannot move. A conventional key-value cache is a stack of tensors whose layout is determined by layer count, head geometry, positional encoding, and the particular weight matrices that produced it. It is large because it stores one state per token per layer, and it is non-portable because those states live in a private basis. Moving it to the client is usually pointless: the payload is comparable to keeping it on the accelerator, and a different model cannot consume it.

A Semantic Object Graph is smaller when $|V| \ll L$ and the edge set remains sparse. Size alone is not enough. If the locally stored object is merely a compressed copy of H , it remains model-bound. Portability requires an explicit split between what is an object and what is a materialization.

5.2 Three-layer cache. The structured cache is factored as $M = (S, C, H)$:

- S is structure: node identities, types, spans, parent links, typed edges, and versions. S is the candidate for a model-agnostic skeleton, analogous to an AST or a typed dependency graph.
- C is canonical semantics: a compact vector c_i per node in a shared embedding space, produced by a small encoder that is not identified with any one LLM. C is the candidate for a portable payload that still carries meaning.
- H is model state: current hidden vectors h_i , edge weights expressed in the serving model’s routing space, and block summaries z_b . H is a scratch materialization for the current weights.

The intended invariant is that S and C may be stored locally and across sessions, while H may be discarded whenever the serving model changes. Cross-model reuse is therefore not “KV translation.” It is lift:

$$H = \text{Lift}_\varphi(C, S).$$

Lift_φ is a model-specific projection, possibly followed by a cheap local repair on uncertain edges. Full token prefill is the fallback when lift is insufficient, not the default path.

5.3 Client object, server compute. Under this split, a request can be written as (q, S, C, Δ) , where q is the new query or edit and Δ is the client-declared dirty set. The server (i) lifts C into H for the current weights, (ii) extends invalidation with $I = \text{Desc}_S(\Delta) \cup \Delta$, (iii) runs GLE and, if needed, PGR only on I , and (iv) returns tokens together with a versioned patch $\Delta S, \Delta C$. The client merges the patch. Long-lived context need not occupy HBM between calls.

This inverts the usual prefix-cache design. Prefix caches keep tensors near the weights because the tensors are huge. SOGR keeps objects near the user because the objects are structured and, under the sparsity conditions of Section 7, small. The accelerator is then closer to a stateless graph-conditioned executor.

Communication cost is part of the hypothesis. Let s be the serialized size of the portable cache. The architecture is attractive for split execution only when s is far smaller than a token-level KV snapshot and when incremental patches are smaller still. A graph that is sparse in FLOPs but dense in metadata would fail this test.

5.4 Changing models. A new model φ' does not inherit H . It inherits S and C , then computes $H' = \text{Lift}_{\varphi'}(C, S)$. If the new model's type or relation vocabulary differs, a schema map $T(S) \rightarrow S'$ may be applied first. Residual uncertainty is handled by PGR rather than by pretending that two models share a hidden basis. The only efficiency claim that matters here is comparative: lift plus local repair must beat full prefill on the original token sequence for a useful class of documents and model pairs. If it does not, the portable cache remains a storage convenience and is not a computational one.

5.5 What is not claimed. Raw per-layer keys and values are not assumed to be linearly alignable across models. Ontology drift is not assumed to be trivial: a code model and a dialogue model may induce different node granularities from the same text, in which case S must be rebuilt or reconciled. Client-supplied edges are not assumed to be trusted; a server that executes a graph it did not construct needs a validation or repair policy. These caveats are part of the hypothesis, not afterthoughts.

6. Mathematical Formulation

Let G_t denote the graph at global repair step t . The global constructor is $G_t = C_\theta(X_t, G_{t-1})$, where C_θ may use full attention. Between repairs, K linear updates are applied: $H_{t,k+1} = F_\varphi(H_{t,k}, G_t, M_t)$, where M_t denotes cached state.

A generic graph-conditioned linear update can be written as:

$$H' = \sigma(HW_0 + P_G(H)W_1), \quad P_G(H)_i = \sum_{j \in N(i)} A_{ij}H_j.$$

Here P_G is a graph propagation operator. If the graph contains E active edges, the propagation term is $O(Ed)$ for hidden width d , ignoring implementation-specific projection costs. The central requirement is that E be substantially smaller than L^2 and that graph maintenance not erase the savings.

For block routing, let b index blocks and z_b be a block summary. Define a stability score $s_b = g(z_b, q)$, where q is the current query/state. A block is skipped when $s_b < \tau$ and its dependency version has not changed. The active computation becomes a function of the active block set B_{active} rather than all B blocks.

The cache invalidation rule can be expressed as $I_{t+1} = \text{Desc}_G(\Delta_t) \cup \Delta_t$, where Δ_t is the set of changed nodes and Desc_G denotes their dependent descendants. This gives an explicit target for incremental computation: recompute only I_{t+1} , subject to a safety policy that can force global repair when invalidation grows beyond a threshold.

For the portable cache, write $M = (S, C, H)$ as in Section 5. Serving with a fixed model is $H_t = \text{Lift}_\varphi(C_t, S_t)$ followed by the updates above. Serving with a new model is $H' = \text{Lift}_{\varphi'}(C, T(S))$, optionally followed by one repair pass. T is the identity when the relation schema is shared.

7. Conditional Complexity Analysis

The proposal does not justify a blanket claim of $O(L)$ complexity. If global graph construction occurs every K layers, the amortized cost contains a global term. Let $c_g L^2$ be the cost of one global pass, $c_e E$ the cost of one graph execution, and $c_m I$ the cost of maintaining/invalidation, where I is the number of affected nodes or edges. Over K execution steps, a simplified average cost is:

$$C_{\text{avg}} \approx \frac{c_g L^2}{K} + c_e E + \frac{c_m I}{K}.$$

The architecture is advantageous when this quantity is lower than the corresponding dense-attention baseline while preserving model quality. If $E = \rho L^2$, then the graph term is still quadratic and the architecture may provide little asymptotic benefit. The desired regime is therefore $\rho \ll 1$, large K , and small average invalidation size I .

This condition is important. The proposal should not be presented as “Transformer becomes $O(L)$ ” without qualification. Its intended benefit is amortized and conditional: expensive global relationship discovery is reused, while stable or unrelated regions are not repeatedly recomputed.

A further potential benefit appears during incremental decoding or editing. If only a small semantic region changes, the relevant cost may scale with the affected dependency subgraph rather than with the complete context, provided the graph remains valid and the task permits such locality.

Split execution adds a communication term $c_s s$, where s is the size of $S \cup C$ or of a patch. Cross-model reuse replaces a prefill cost on the order of $c_g L^2$ with a lift cost on the order of $c_\ell |V|$ plus any repair. That substitution is advantageous only when $c_\ell |V| + c_{\text{repair}} \ll c_g L^2$ at matched quality. The same warning as above applies: this is a regime, not an identity.

8. Relationship to Existing Work

SOGR is related to several established directions but is not identical to any single one. Linear attention reformulates attention to obtain linear sequence complexity and exposes a recurrent/fast-weight interpretation. Mamba and related selective state-space models similarly pursue efficient long-sequence processing while addressing content-dependent memory. Kimi Linear demonstrates that hybrid architectures can interleave linear attention with periodic global attention and achieve strong long-context efficiency.

Sparse attention methods reduce computation by restricting the set of interactions, often using fixed windows, learned routing, hashing, or chunk selection. The conceptual distinction proposed here is that the interaction structure is treated as a persistent intermediate representation rather than merely a mask for one attention operation.

The closest conceptual relatives outside LLMs are incremental compilers, databases with materialized views, dependency-based build systems, and reactive UI frameworks. In each case, a dependency structure allows local changes to propagate without recomputing unrelated state. The research question is whether the same computational principle can be made reliable for language representations, where dependencies are uncertain and context-sensitive.

On the systems side, paged and prefix KV caches keep model-bound tensors near the weights. Semantic caches and retrieval-augmented memory keep documents or embeddings that a model can reread, but they are not an executable dependency graph for incremental hidden-state update. Representation-alignment methods can map vectors between spaces; they do not by themselves yield a typed, versioned object that a runtime can invalidate. SOGR’s portable-cache claim is the combination: a client-held IR whose structure indexes computation and whose canonical vectors can be lifted, rather than a blob of keys and values that must be stored where the GPU is.

This manuscript therefore claims an architectural hypothesis, not priority over all constituent techniques. A formal novelty/priority determination would require a systematic literature and patent search beyond the scope of this draft.

9. Training Strategy

A practical training strategy is to start from a conventional Transformer or hybrid linear-attention model and introduce auxiliary supervision for graph quality. Candidate graph edges can be derived from attention patterns, coreference links, dependency parses, latent probes, or learned edge predictors. The graph should ultimately be trainable end-to-end rather than requiring manual annotation.

One possible objective is $L = L_{\text{lm}} + \lambda_r L_{\text{relation}} + \lambda_s L_{\text{stability}} + \lambda_i L_{\text{invalidation}}$. L_{lm} is the ordinary language-modeling loss. L_{relation} encourages useful dependency prediction. $L_{\text{stability}}$ encourages graph persistence when the semantic state is unchanged. $L_{\text{invalidation}}$ penalizes unnecessary recomputation or incorrect locality assumptions.

A curriculum can begin with fixed graphs or teacher-generated graphs, then progressively remove external structural supervision. This allows the model to learn the runtime semantics before being asked to discover the structure independently.

If portable cache is trained at all, Lift_φ should be supervised against hidden states obtained from a full forward pass on the same document, while a reconstruction or stability term keeps C informative without collapsing into a copy of H . Cross-model lift can be trained on paired materializations of the same (S, C) under two frozen backbones. These objectives are optional extensions of the runtime, not prerequisites for testing incremental recomputation.

10. Experimental Program

The first experiment should not train a billion-parameter model. A small controlled model is sufficient to test the central hypothesis. The key comparison is not only perplexity or benchmark score but compute per useful update and dependency locality.

Recommended evaluations include:

1. Language modeling perplexity at equal FLOPs
2. Long-context retrieval such as MQAR and needle-in-a-haystack variants
3. Exact-copy and associative recall tests
4. Semantic-edit tests in which one entity or fact is modified and the required recomputation region is measured
5. Graph precision/recall against automatically generated dependency probes
6. End-to-end GPU throughput using block-sparse kernels
7. Ablations over graph refresh frequency K , block size, graph density ρ , and cache invalidation threshold τ

A decisive experiment is an edit locality benchmark. Construct documents with explicit dependency chains. Change one fact in the middle, then measure how many hidden states must be recomputed to recover the same output quality as a full recomputation. The central hypothesis predicts that the affected subgraph will remain substantially smaller than the complete context for a meaningful class of edits.

The final in-model experiment should compare SOGR against full attention, a strong linear-attention baseline, a standard hybrid architecture, and a chunk-sparse attention baseline under matched parameter count, training tokens, hardware, and quality targets.

Portable-cache experiments are separate and can be run even on small models:

8. Serialized size of $S \cup C$ versus token-level KV for the same context length
9. Quality and FLOPs of $\text{Lift}_\varphi(C, S)$ versus full prefill on the original tokens
10. Cross-model transfer: store (S, C) under model φ , lift into φ' , and compare against prefill on φ'
11. Patch size and round-trip latency for client-held cache with only a local edit
12. Schema mismatch: transfer graphs between models with different node granularity and measure how often PGR must rebuild S

A decisive negative result for the deployment hypothesis is easy to state. If lift cannot approach prefill quality without rereading most tokens, or if s is not substantially smaller than KV, then the object graph remains an internal IR and should not be advertised as a client-side working memory.

A CPU-scale baseline is included because the author currently has only a non-gaming notebook (CPU, no CUDA). That hardware is the experimental floor, not the intended scale of SOGR.

In experiments/llm_to_object, DistilGPT-2 (82M) on CPU contracts a 156-token paragraph into 54 nodes and 115 edges. Serialized $S \cup C$ is 31 KB versus a 5.6 MB KV snapshot (about $184 \times$ smaller on this toy). A one-place edit Shanghai $\rightarrow$ Beijing dirties 3.7% of nodes at one hop and 40.7% under transitive closure, versus 100% of tokens for full prefill. Size and objectification therefore hold at this floor; one-hop locality holds; transitive locality is only partial, because an attention graph is denser than a linguistic dependency graph.

In experiments/compiler_lite, the same serialized graph is fed as the sole context to two 0.5B instruction models (Qwen2.5-0.5B-Instruct and Qwen2-0.5B-Instruct), against feeding the raw paragraph. Full text scores 6/6 on a six-item factual quiz for both models; the object graph scores 3/6. This is compiler-lite (S as readable context), not a KV compiler. It is weak support for portability and a warning that the current extraction/serialization loses answer-critical structure.

These runs do not replace GPU-scale perplexity, long-context retrieval, or true $S + C \rightarrow$ KV lift. They are the maximum that the present machine can honestly support.

11. Limitations and Failure Modes

Natural language dependencies are not deterministic AST edges. Pronouns, discourse relations, negation, irony, world knowledge, and long-range semantic interactions can invalidate an apparently local graph. A safe system therefore needs uncertainty-aware edges and mechanisms for forcing global repair.

Graph construction itself can be expensive. If the constructor must repeatedly perform nearly full attention, the expected savings can disappear. The architecture consequently depends on a sufficiently long interval between graph repairs and a graph that remains useful during that interval.

Hardware sparsity is another limitation. Arbitrary graph traversal can be slower than dense matrix multiplication on GPUs. Block-level grouping is therefore not an optional implementation detail; it is part of the architectural hypothesis. The graph should expose contiguous, batchable regions whenever possible.

Finally, a smaller computation graph does not imply identical model quality. The graph may discard a weak but important long-range dependency. The system must therefore optimize for safe approximation, not merely maximum sparsity.

Portable cache adds further failure modes. Hidden states are not a shared coordinate system; attempting to map H across models is likely to fail even when S is reusable. Canonical vectors C may be too lossy, so that lift is cheap and wrong. Relation schemas may not align, so “conversion” becomes re-parsing. A client-held graph is a high-value personal artifact and a possible injection surface if the server executes untrusted edges. None of these issues is solved by sparsity alone.

A further limitation is computational budget. All measurements in this draft were obtained on a CPU laptop without a discrete GPU. 0.5B-class instruction models and DistilGPT-2 are the present ceiling. Results should be read as a floor, not as evidence about 7B-class models or 16K-128K context.

12. Conclusion

SOGR proposes a shift in the computational abstraction of language models. Instead of treating the token sequence as the only persistent state and rediscovering relationships at every layer, the architecture introduces a Semantic Object Graph as a cacheable intermediate representation. Global attention constructs and repairs that representation; graph-conditioned linear operators perform repeated propagation; structured caches retain stable state; and hierarchical block routing avoids unnecessary computation.

The proposal is intentionally falsifiable. Its success depends on measurable conditions: graph sparsity, graph stability, invalidation locality, constructor frequency, and hardware efficiency. If those conditions fail, the architecture reduces to an expensive hybrid attention system and offers little advantage. If they hold, the same principle could provide a route toward incremental language-model runtimes in which the unit of computation is not a token sequence but a dependency subgraph.

The central research question is therefore not “how can Transformer attention be made slightly cheaper?” but “can language be compiled into a stable computational structure that allows the model to update only what changed?” A follow-on question is whether that structure can leave the accelerator: whether the user can hold the object, the server can hold only the weights, and a new model can lift the object instead of inheriting a KV tensor. SOGR is a concrete architectural hypothesis for testing both questions.

13. Claimed Contributions of This Draft

- A Semantic Object Graph Runtime abstraction for language models
- A separation between low-frequency global relationship construction and high-frequency graph-conditioned execution
- A structured cache and dependency-version mechanism for incremental recomputation
- A three-layer cache (S, C, H) that separates portable structure and canonical semantics from model-bound activations
- A split-runtime hypothesis in which the client stores the object graph and the server performs graph-conditioned computation
- A cross-model reuse hypothesis based on lifting (S, C) rather than translating raw KV tensors

- A hierarchical block-diff mechanism designed to align semantic sparsity with GPU-friendly contiguous computation
- A conditional complexity model and experimental protocol that can falsify both the in-model efficiency advantage and the portable-cache deployment claim
- A laptop CPU baseline showing objectification and KV-size reduction on DistilGPT-2, partial edit locality, and a 50% drop when two 0.5B models must answer from S alone

14. References

1. Vaswani, A. et al. *Attention Is All You Need*. NeurIPS, 2017.
2. Katharopoulos, A., Vyas, A., Pappas, N., Fleuret, F. *Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention*. ICML/PMLR 119, 5156–5165, 2020.
3. Schlag, I., Irie, K., Schmidhuber, J. *Linear Transformers Are Secretly Fast Weight Programmers*. ICML/PMLR 139, 9355–9366, 2021.
4. Kitaev, N., Kaiser, Ł., Levskaya, A. *Reformer: The Efficient Transformer*. ICLR, 2020.
5. Gu, A., Dao, T. *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*. arXiv:2312.00752, 2023.
6. Kimi Team et al. *Kimi Linear: An Expressive, Efficient Attention Architecture*. arXiv:2510.26692, 2025.
7. Veličković, P. et al. *Graph Attention Networks*. ICLR, 2018.
8. Kim, Y., Denton, C., Hoang, L., Rush, A. M. *Structured Attention Networks*. ICLR, 2017.
9. Loynd, R. et al. *Working Memory Graphs*. ICML/PMLR 119, 6404–6414, 2020.
10. Dao, T. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS, 2022.
11. Dao, T. et al. *FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning*. ICLR, 2024.
12. Beltagy, I., Peters, M. E., Cohan, A. *The Long-Document Transformer*. arXiv:2004.05150, 2020.
13. Child, R. et al. *Generating Long Sequences with Sparse Transformers*. arXiv:1904.10509, 2019.
14. Liu, Y. et al. *Blockwise Parallel Transformer for Long Context Modeling*. Related line of chunk/block efficient sequence modeling; see contemporary long-context literature.
15. McKinley, K. S. et al. *Compiler and incremental computation literature on dependency-directed recomputation; the analogy motivates the runtime design but is not claimed as a new compiler technique*.

14. Originality and attribution note

This manuscript presents the SOGR architecture as a research proposal derived from the author’s own conceptual development. It intentionally distinguishes the proposed combination and system-level abstraction from prior work on linear attention, sparse attention, graphs, caches, hybrid architectures, prefix KV stores, and representation alignment. The paper does not claim that every individual component is novel, nor does it claim proven priority without a comprehensive prior-art search. All numerical complexity statements are analytical conditions or illustrative examples unless explicitly identified as reported results from cited work. Portable cache, client-side objects, and cross-model lift are stated as deployment hypotheses with explicit falsifiers, not as demonstrated systems properties.